# GD3000

这是一个比较老的二进制文件，它取之于 gameboy 游戏主机里的 rom，我们需要用 bgd 去打开模拟，并且调试。

## 0X00

Key **UP** : 40(decimal); ( 0100 0000)

Key **LEFT:** 20(dec);

Key **RIGHT** : 10

Key **B** :2

Key **A** :1

Instructions：

LDI a,(H1) --->  a=H1, H1+=H1;

CP b  ---> cp a,b

# 0x10

```
call 0253
call 0312
jp   017C
call 0303
call 03BB
ld   a,(C010)
cp   a,00
jr   z,01DD
ld   hl,C001
ld   e,61
call 027A
cp   a,01
jr   nz,01DD
ld   hl,C005
ld   e,62
call 027A
cp   a,01
jr   nz,01DD
ld   hl,C009
ld   e,61
call 027A
cp   a,01
jr   nz,01DD
ld   hl,C00D
ld   e,70
```

当我们运行这个程序观察，它的逻辑会一直卡在这段函数里，很明显这段函数是密码验证函数。

首先它验证了 c010 是否为 0？ 如果为 0，则跳到 0x01dd 中，可以观察到，每次校验失败都会跳到这个地址，可以推测这个地址是错误处理地址。

那么就可以通过这个错误处理推测，c001,c005,c009,c00d 的判断，应该是对密码的校验，然后通过 h1 和 e 进行传参，调用 0x27A 的函数。

# 0X11

为了更快速解题，我打算先不看 0x27A 的校验逻辑了，我直接 patch 这个函数，让这些校验直接可以通过。

Patch 前:

```
ROM0:017C CD 03 03        call 0303          ;6  6
ROM0:017F CD BB 03        call 03BB          ;6  12
ROM0:0182 FA 10 C0        ld   a,(C010)      ;4  16
ROM0:0185 FE 00           cp   a,00          ;2  18
ROM0:0187 28 54           jr   z,01DD        ;2  20
ROM0:0189 21 01 C0        ld   hl,C001       ;3  23
ROM0:018C 1E 61           ld   e,61          ;2  25
ROM0:018E CD 7A 02        call 027A          ;6  31
ROM0:0191 FE 01           cp   a,01          ;2  33
ROM0:0193 20 48           jr   nz,01DD       ;2  35
ROM0:0195 21 05 C0        ld   hl,C005       ;3  38
ROM0:0198 1E 62           ld   e,62          ;2  40
ROM0:019A CD 7A 02        call 027A          ;6  46
ROM0:019D FE 01           cp   a,01          ;2  48
ROM0:019F 20 3C           jr   nz,01DD       ;2  50
ROM0:01A1 21 09 C0        ld   hl,C009       ;3  53
ROM0:01A4 1E 61           ld   e,61          ;2  55
ROM0:01A6 CD 7A 02        call 027A          ;6  61
ROM0:01A9 FE 01           cp   a,01          ;2  63
ROM0:01AB 20 30           jr   nz,01DD       ;2  65
ROM0:01AD 21 0D C0        ld   hl,C00D       ;3  68
ROM0:01B0 1E 70           ld   e,70          ;2  70
ROM0:01B2 CD 7A 02        call 027A          ;6  76
ROM0:01B5 FE 01           cp   a,01          ;2  78
ROM0:01B7 20 24           jr   nz,01DD       ;2  80
```

Patch 后:

```
ROM0:017C CD 03 03        call 0303          ;6  6
ROM0:017F CD BB 03        call 03BB          ;6  12
ROM0:0182 FA 10 C0        ld   a,(C010)      ;4  16
ROM0:0185 FE 00           cp   a,00          ;2  18
ROM0:0187 20 54           jr   nz,01DD       ;2  20
ROM0:0189 21 01 C0        ld   hl,C001       ;3  23
ROM0:018C 1E 61           ld   e,61          ;2  25
ROM0:018E CD 7A 02        call 027A          ;6  31
ROM0:0191 FE 01           cp   a,01          ;2  33
ROM0:0193 28 48           jr   z,01DD        ;2  35
ROM0:0195 21 05 C0        ld   hl,C005       ;3  38
ROM0:0198 1E 62           ld   e,62          ;2  40
ROM0:019A CD 7A 02        call 027A          ;6  46
ROM0:019D FE 01           cp   a,01          ;2  48
ROM0:019F 28 3C           jr   z,01DD        ;2  50
ROM0:01A1 21 09 C0        ld   hl,C009       ;3  53
ROM0:01A4 1E 61           ld   e,61          ;2  55
ROM0:01A6 CD 7A 02        call 027A          ;6  61
ROM0:01A9 FE 01           cp   a,01          ;2  63
ROM0:01AB 28 30           jr   z,01DD        ;2  65
ROM0:01AD 21 0D C0        ld   hl,C00D       ;3  68
ROM0:01B0 1E 70           ld   e,70          ;2  70
ROM0:01B2 CD 7A 02        call 027A          ;6  76
ROM0:01B5 FE 01           cp   a,01          ;2  78
ROM0:01B7 28 24           jr   z,01DD        ;2  80
```

Screen output:

这说明我的逻辑是对的，确实弹出来一个显示框，但是并没有我们可以识别到的 flag 值，这说明，它很有拿着我们的参数进行了计算，因为我们只是 patch 了逻辑，什么都没有输入，导致它的并不能正确恢复出 Flag 图片。看来我们还是得逆向 0x27A 这个函数。

# 0X12

```
ROM0:027A 2A          ldi  a,(hl)        ;2  9
ROM0:027B 47          ld   b,a           ;1  10
ROM0:027C 7E          ld   a,(hl)        ;2  12
ROM0:027D B8          cp   b             ;1  13
ROM0:027E 20 11       jr   nz,0291       ;2  15
ROM0:0280 FE 40       cp   a,40          ;2  17
ROM0:0282 20 0D       jr   nz,0291       ;2  19
ROM0:0284 2A          ldi  a,(hl)        ;2  21
ROM0:0285 47          ld   b,a           ;1  22
ROM0:0286 2A          ldi  a,(hl)        ;2  24
ROM0:0287 B0          or   b             ;1  25
ROM0:0288 47          ld   b,a           ;1  26
ROM0:0289 2A          ldi  a,(hl)        ;2  28
ROM0:028A B0          or   b             ;1  29
ROM0:028B BB          cp   e             ;1  30
ROM0:028C 20 03       jr   nz,0291       ;2  32
ROM0:028E 3E 01       ld   a,01          ;2  34
ROM0:0290 C9          ret                ;4  38
ROM0:0291 3E 00       ld   a,00          ;2  40
ROM0:0293 C9          ret                ;4  44
ROM0:0294 1A          ld   a,(de)        ;2  46
ROM0:0295 22          ldi  (hl),a        ;2  48
ROM0:0296 13          inc  de            ;2  50
```

可以观察到这段函数的逻辑为， 0x291 为错误处理函数，那么其它的分支就是正确的处理函数。

a=$1, b=a=$1, a=$2, (a==b) =1， $2==40，$2==$1 == 40

a=$3， b=$3, a=$4, a= $3 or $4, b=$3 or $4, a =$5, a=$5 or $4 or $3

If a==e , a=1 else a=0. 由此可以知道我们要让 $5 or $4 or $3 ==e。

因为每个 0x1,0x5,0x9,0xd 都要走这个函数逻辑，由此我们可以知道，

图像至少是

**Up up ? ?**

**Up up ? ?**

**Up up ? ?**

**Up up ? ?**

然后我从主函数可以知道，

```
ROM0:017F CD BB 03        call 03BB          ;6  6
ROM0:0182 FA 10 C0        ld   a,(C010)      ;4  10
ROM0:0185 FE 00           cp   a,00          ;2  12
ROM0:0187 28 54           jr   z,01DD        ;2  14
ROM0:0189 21 01 C0        ld   hl,C001       ;3  17
ROM0:018C 1E 61           ld   e,61          ;2  19
ROM0:018E CD 7A 02        call 027A          ;6  25
ROM0:0191 FE 01           cp   a,01          ;2  27
ROM0:0193 20 48           jr   nz,01DD       ;2  29
ROM0:0195 21 05 C0        ld   hl,C005       ;3  32
ROM0:0198 1E 62           ld   e,62          ;2  34
ROM0:019A CD 7A 02        call 027A          ;6  40
ROM0:019D FE 01           cp   a,01          ;2  42
ROM0:019F 20 3C           jr   nz,01DD       ;2  44
ROM0:01A1 21 09 C0        ld   hl,C009       ;3  47
ROM0:01A4 1E 61           ld   e,61          ;2  49
ROM0:01A6 CD 7A 02        call 027A          ;6  55
ROM0:01A9 FE 01           cp   a,01          ;2  57
ROM0:01AB 20 30           jr   nz,01DD       ;2  59
ROM0:01AD 21 0D C0        ld   hl,C00D       ;3  62
ROM0:01B0 1E 70           ld   e,70          ;2  64
ROM0:01B2 CD 7A 02        call 027A          ;6  70
ROM0:01B5 FE 01           cp   a,01          ;2  72
ROM0:01B7 20 24           jr   nz,01DD       ;2  74
ROM0:01B9 CD 0C 03        call 030C          ;6  80
```

那么就可以由此通关我们推导出来的$5 or $4 or $3 ==e。计算出，分布图为

**Up up left A**

**Up up left b**

**Up up left a**

**Up up left right**

Screen output:

Ok